A Representation Reversal Under Quantum-Circuit Size Shift: Wire-Dependency DAGs versus Canonical Layer Grids

Anonymous Authors
Affiliation withheld for review

November 2, 2026

Abstract

Quantum-circuit learning is often developed on small circuits because simulation and labeled-data generation become expensive with scale. This makes extrapolation across circuit size a central representation problem. We isolate that problem with a controlled probe: predict which gates lie on a circuit's maximum wire-dependency path. The labels are exact, the two models receive identical gate and operand features, and their parameter counts differ by less than one percent. A canonical layer-grid GRU is much better when train and test sizes match (0.9406 versus 0.8556 AUROC), so ordinary in-distribution model selection would choose it. That decision reverses on wider and longer circuits: after training on 4–8 qubits and 24–56 gates, a wire-dependency DAG network reaches 0.8078 AUROC and 0.6341 F1 on 9–12 qubits and 64–96 gates, compared with 0.7861 and 0.6055 for the grid. The DAG wins all three frozen paired seeds. Our contribution is a controlled demonstration that the circuit encoding preferred in distribution can be the worse encoding under size shift. The result supports explicit dependency structure as an inductive bias for size-generalizing circuit models; it does not claim a new critical-path algorithm or a production compiler optimizer.

1 The contribution in one result

A quantum circuit has a visual layout, an operation sequence, and a dependency graph. These encodings contain related information, but they make different regularities easy for a neural model to learn. A layer grid exposes stable coordinates such as "wire 2, layer 5." A wire-dependency DAG exposes relations such as "this CX follows the previous operation on its control wire." Grid coordinates change as width and depth grow; the meaning of a predecessor edge does not.

This paper tests one hypothesis:

> **If a prediction depends on wire dependencies, then explicitly encoding those dependencies should transfer better from small circuits to larger circuits, even when a positional grid fits the training distribution better.**

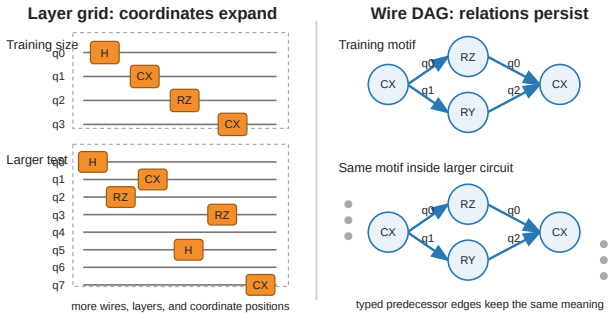


Figure 1: Why the encodings extrapolate differently. Adding wires and independent operations expands and shifts grid coordinates. The local wire-dependency motif retains the same typed predecessor relations. The experiment asks which inductive bias survives this size change.

The experiment produces a representation reversal. On same-size circuits, the grid leads by 0.0850 AUROC. On larger circuits, the DAG leads by 0.0217 AUROC and 0.0286 F1, winning every paired seed. A practitioner using only same-size validation would therefore select the model that generalizes worse to larger circuits.

The contribution is an empirical result about *representation and extrapolation*. Depth-critical labels are exactly computable, so the learned model is not intended to replace the critical-path algorithm. Exact labels instead provide a clean probe: supervision has no simulator noise, and success requires combining information along precisely the relations the DAG represents. This separates representation behavior from approximation error in a quantum simulator or compiler oracle.

2 Why size transfer matters

Quantum machine-learning systems frequently train on smaller instances than those of ultimate interest. Classical simulation cost grows rapidly with qubit count, hardware data are limited, and compiler searches can be expensive. Frameworks for quantum machine learning

make circuit data accessible [1], but a model that interpolates among familiar sizes may still rely on coordinates that do not persist at deployment scale.

The issue is especially relevant to compilation. Circuit depth, two-qubit gates, and routing overhead constrain near-term workloads [2, 3, 4]. Many learned compiler components consume a circuit encoding before predicting cost, selecting transformations, or guiding search. Their first design choice is therefore representational: flatten the circuit, place it on a grid, or preserve dependencies as a graph.

Graph message passing supplies a relational inductive bias [5, 6]. It can also be harder to optimize and may discard useful global coordinates. We should not expect a graph to win everywhere. The scientifically useful question is where its bias helps and what an in-distribution benchmark would miss.

Our study contributes:

1. a matched DAG-versus-grid experiment that changes circuit size rather than merely resampling circuits of the same shape;
2. a frozen three-seed result showing that representation ranking reverses between interpolation and size extrapolation;
3. a mechanism check for legal topological reordering and strong sequence, Transformer, canonical-grid, and recurrent-DAG development controls;
4. a complete record of protocols, split hashes, negative trials, and follow-up tasks that did not establish a practical optimizer advantage.

3 Controlled representation probe

3.1 Wire-dependency labels

Let a circuit contain operations $V = \{v_i\}_{i=1}^{m}$ on n logical qubits. For every qubit wire, connect consecutive operations that touch that wire. Their union is a directed acyclic graph $G = (V, E)$. Each edge retains its wire identity. Independent operations have no edge between them, even if one happens to be serialized first.

Let $f(v)$ be the longest predecessor path ending at v, $b(v)$ the longest successor path beginning at v, and $D = \max_v f(v)$. The exact node label is

$$y_v = \mathbf{1}[f(v) + b(v) - 1 = D]\,. \tag{1}$$

Thus, $y_v = 1$ when gate v belongs to at least one maximum dependency path. The label is a scheduling property of the complete circuit, not a local gate attribute.

Consider two chains that merge at a CX: $H(q_0) \to CX(q_0, q_1)$ and $R_Z(q_2) \to R_Y(q_2)$, followed by $CX(q_1, q_2)$. Operations on the two chains can be interleaved in several legal sequences. Their indices and grid occupancy vary, while their typed predecessor relations and critical-path calculation remain unchanged. Larger circuits add more such independent and merging chains.

3.2 Matched encoders

Every operation receives the same features in both models: one-hot gate type, membership of each wire, CX control and target roles, sine and cosine of the rotation angle, and arity. The maximum feature width supports the full 12-qubit evaluation range.

The **DAG model** projects features to width 64 and applies six residual bidirectional message-passing blocks. Forward messages come from predecessors, backward messages come from successors, and the wire identity is concatenated to every message. A linear head predicts one logit per gate.

The **grid model** computes deterministic earliest dependency layers, then orders gates within each layer by operands, type, and parameters. A two-layer bidirectional GRU of width 96 processes this canonical sequence and predicts the same node labels. Canonicalization makes this a strong control: it removes arbitrary serialization noise rather than giving the DAG an artificially weak sequence baseline.

The DAG has 87,745 trainable parameters and the grid 88,417, a 0.77% difference. Both receive 6,000 updates, the same batches, AdamW optimizer, node features, labels, and validation-only checkpoint selection.

4 Experimental firewall

Deterministic circuits contain H, RY, RZ, and directed CX operations. Training and validation use 4–8 qubits and 24–56 gates. The in-distribution test uses the same range with a fresh seed. The size-shifted test uses 9–12 qubits and 64–96 gates. Training, validation, same-size test, reordered test, and larger test have distinct stored SHA-256 hashes.

Development considered raw bidirectional sequence, token Transformer, canonical-grid GRU, fixed-depth DAG, and full-depth recurrent-DAG encoders. The final comparison froze the selected six-layer DAG and canonical grid, all data ranges, 6,000 updates, metrics, and success rule before final data generation. Three paired seeds (51, 52, and 53) train on identical examples.

The predeclared size-transfer claim required (i) positive DAG-minus-grid larger-circuit AUROC for every seed, (ii) mean improvement of at least 0.015 AUROC, and (iii) positive mean F1 improvement. Same-size superiority was not required. We report node-level AUROC and F1, with mean and sample standard deviation across seeds.

5 The representation ranking reverses

Table 1 shows the reversal. The grid is substantially better on the same size distribution. This is evidence that its canonical coordinates are useful, not that it is a weak

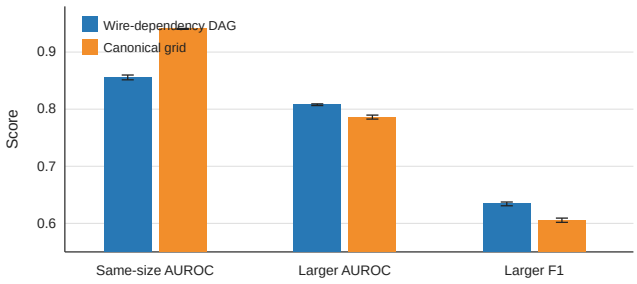


Figure 2: The central result. In-distribution evaluation selects the grid; larger-circuit evaluation selects the DAG. Error bars are sample standard deviations over three paired seeds.

Table 1: Frozen final results (mean ± sample standard deviation).

Encoding	Same AUROC	Larger AUROC	Larger F1
Wire DAG	$0.8556 \pm .0042$	$\mathbf{0.8078 \pm .0015}$	$\mathbf{0.6341 \pm .0033}$
Layer grid	$\mathbf{0.9406 \pm .0008}$	$0.7861 \pm .0035$	$0.6055 \pm .0037$

baseline. Under size shift, however, paired DAG-minus-grid AUROC differences are +0.0202, +0.0177, and +0.0274. Their mean is +0.0217, exceeding the frozen +0.015 gate. Larger-circuit F1 improves by +0.0286 on average. Every seed favors the DAG on both shifted metrics.

Descriptive seed-level 95% t intervals are [0.0093, 0.0342] for the AUROC difference and [0.0219, 0.0353] for the F1 difference. With three seeds, these intervals describe consistency rather than supplying strong asymptotic inference. The all-seed result and frozen effect-size criterion are the primary evidence.

5.1 What explains the result?

The grid can learn that certain canonical positions are commonly critical within the training size range. Wider and longer circuits change the number of positions, chain lengths, and merge locations. The bidirectional GRU can process the longer sequence, but it must extrapolate its positional patterns over many more steps.

The DAG performs the same six local dependency updates regardless of total serialization length. A predecessor message still means "the previous operation on this wire" on a 12-qubit circuit. This does not make the model an exact longest-path solver; six layers cannot propagate across an arbitrarily long graph. It does make its learned features depend on relations whose semantics remain stable when the circuit grows.

A legal topological-reordering test supports this mechanism: DAG logits agree within 10^{-6} across reorderings. The canonical grid is also stable because it explicitly re-canonicalizes. A raw sequence control is not stable and reached only 0.7303 AUROC on larger development circuits, compared with 0.7687 for the canonical grid and 0.7933 for the selected DAG at that development stage. A token Transformer and recurrent DAG were weaker development controls. These development comparisons motivated the final architectures; only Table 1 is confirmatory.

6 What the result does and does not mean

The confirmed statement is:

> For depth-critical gate prediction under the tested 4–8 to 9–12 qubit shift, explicit wire-dependency message passing transfers better than a parameter- and update-matched canonical layer grid, even though the grid is much more accurate in distribution.

This establishes neither universal DAG superiority nor a new quantum compiler. Critical paths are exactly computable in linear time, so using a neural network for this label alone would be unnecessary. The value of the task is experimental control: it reveals whether an encoder learns dependency structure that survives a size change.

We attempted three downstream checks. Fidelity-safe single-gate deletion was saturated by a smallest-angle heuristic. Controlled identity-rewrite prioritization favored DAG in all three seeds (51.94% versus 48.15% oracle depth recovered) but missed its predeclared five-point effect-size gate. A structured routed-rewrite development task used verified safe templates from QAOA-like, VQE, arithmetic, and chemistry-inspired circuits; its candidate-aware grid remained stronger (6.664 versus 6.305 mean routed-CX reduction), so no final routed test was opened. These failures prevent a practical optimizer claim. They do not alter the frozen representation result, which asks a different question.

The next practical test should predict a cost that is expensive to compute directly from the source DAG, such as post-routing performance over many transformations, calibrated noise-aware utility, or synthesis interactions. It should use public circuit families, several hardware graphs, and enough repeated workloads to amortize training cost.

7 Limitations and reproducibility

The confirmatory circuits are synthetic, use four gate types, contain at most 12 qubits, and use only three final seeds. The task probes one topological property. Parameter counts and updates are matched, but neither model receives an exhaustive independent hyperparameter search. The native PyTorch DAG is source-informed by LayerDAG-style message passing; DGL runtime parity is not claimed.

Reproducibility artifacts include the pre-final protocol, deterministic generators, split hashes, training histories, parameter counts, every development and final metric, failed downstream studies, and a compute ledger. Twenty-eight tests cover gate semantics, canonical scheduling, graph aggregation, topological equivariance, fidelity labels, and exact rewrite templates. The complete internally timed CPU training lower bound is 153.82 minutes; no GPU or paid resource was used.

8 Conclusion

The central lesson is methodological: evaluating quantum-circuit encoders only on familiar sizes can reverse the model-selection decision needed for deployment on larger circuits. A canonical layer grid fits the training regime best, while a wire-dependency DAG preserves more useful structure across width and length changes. This controlled result makes explicit dependency structure a justified candidate for size-generalizing quantum-circuit models. Broader real-circuit and hardware-aware studies are required before extending that conclusion to practical compiler optimization.

References

[1] Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J. Martinez, Jae Hyeon Yoo, Sergei V. Isakov, Philip Massey, Ramin Halavati, Murphy Yuezhen Niu, Alexander Zlokapa, Evan Peters, Owen Lockwood, Andrea Skolik, Sofiene Jerbi, Vedran Dunjko, Martin Leib, Michael Streif, David Von Dollen, Hongxiang Chen, Shuxiang Cao, Roeland Wiersema, Hsin-Yuan Huang, Jarrod R. McClean, Ryan Babbush, Sergio Boixo, Dave Bacon, Alan K. Ho, Hartmut Neven, and Masoud Mohseni. Tensorflow quantum: A software framework for quantum machine learning. *arXiv preprint arXiv:2003.02989*, 2021.

[2] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[3] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the qubit mapping problem for NISQ-era quantum devices. *ASPLOS*, pages 1001–1014, 2019.

[4] Alexander Cowtan, Silas Dilkes, Ross Duncan, Will Simmons, and Seyon Sivarajah. On the qubit routing problem. *Proceedings of the 14th Conference on the Theory of Quantum Computation, Communication and Cryptography*, 2019.

[5] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.

[6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.